

PROGETTAZIONE APP REACT

IoT4Care - monitoraggio del sonno

SOMNI

Checchetti Christian, VR501045

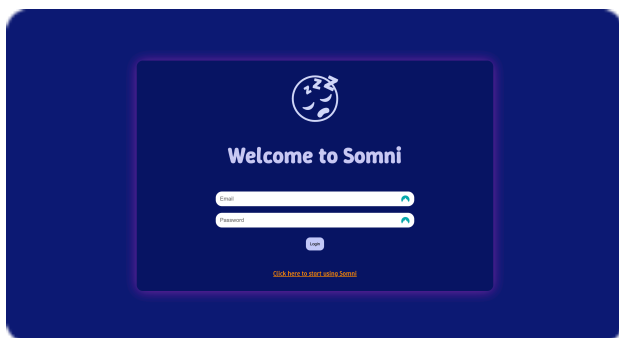
Con la seguente relazione si intende descrivere il processo costruttivo e il funzionamento dell'app di monitoraggio del sonno che ho realizzato come progetto inerente al corso di progettazione di app React, legato alle proposte fornite dal gruppo di ricerca IoT4Care.

STRUTTURA BASE - FRONTEND:

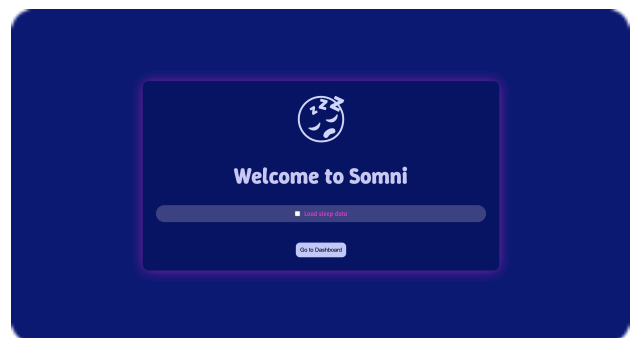
Per quanto riguarda il frontend, concentrandosi sull'interfaccia utente quindi, *Somni* si presenta all'utente con una schermata di accesso semplice che consente di accedere in caso di registrazione preesistente o di registrarsi in caso di primo accesso al servizio.

Una volta inserite le credenziali, si accede a una schermata simile alla precedente dal punto di vista grafico, che consente di scegliere se caricare nuovi dati relativi alle sessioni di sonno dell'utente prima di accedere alla dashboard.

Nel caso in cui si scelga di caricare dei dati prima di procedere, il pulsante di reindirizzamento alla dashboard risulterà inutilizzabile fino al completamento dell'operazione di caricamento.



Login



Upload dei dati del sonno

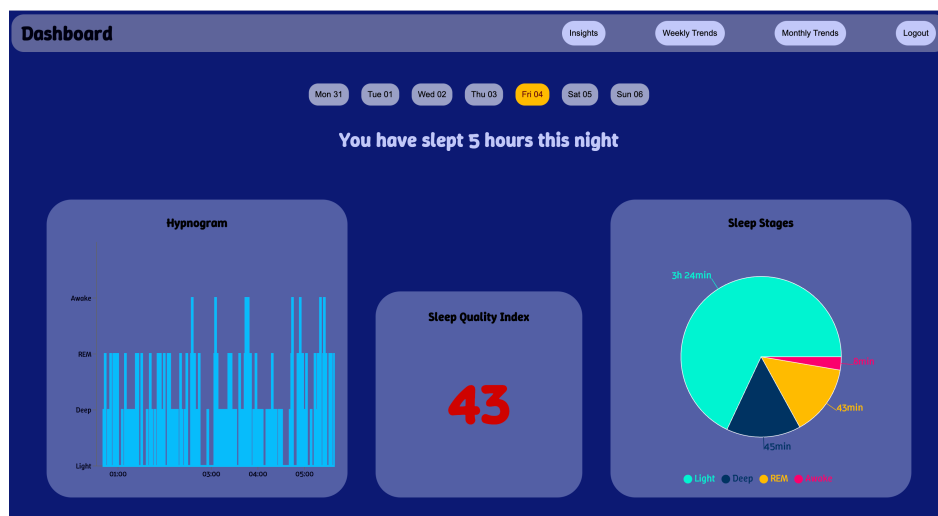
Si procede quindi verso la dashboard, il cuore pulsante dell'applicazione, dove sono raccolti i dati principali relativi alle sessioni di sonno.

Nella parte superiore dello schermo è presente una barra di navigazione che fornisce l'accesso alle sezioni complementari dell'applicazione e al pulsante di logout, che consente di uscire dal servizio e reindirizza l'utente alla pagina di accesso.

Scendendo, seguendo un approccio top-down, si incontra un selettore semplice ma elegante che permette di selezionare il giorno della settimana per il quale visualizzare i dati, in riferimento alla settimana in corso. Al primo accesso alla dashboard, verrà selezionato il giorno corrente.

Sotto questo selettore compare un messaggio che indica all'utente le ore di sonno dormite nella notte selezionata.

Segue la parte principale della dashboard, situata nella parte inferiore dello schermo, costituita da tre card. Quella di sinistra presenta un ipnogramma che consente di verificare lo sviluppo del sonno nelle sue quattro fasi principali: **sonno leggero, sonno profondo, veglia e fase REM**. La card centrale è dedicata all'indice di qualità del sonno, un numero compreso tra 0 e 100 che tiene conto del numero di ore dormite e del tempo trascorso nelle varie fasi per determinare la qualità del riposo dell'utente. Il valore mostra un colore differente a seconda del range di appartenenza dell'indice, spaziando da verde, giallo, rosso e viola; in relazione rispettivamente a un ottimo sonno, un sonno buono ma migliorabile, scarso e decisamente insufficiente. Infine, sulla destra, è possibile visualizzare un grafico a torta che indica il tempo complessivo trascorso in ogni fase.



Dashboard

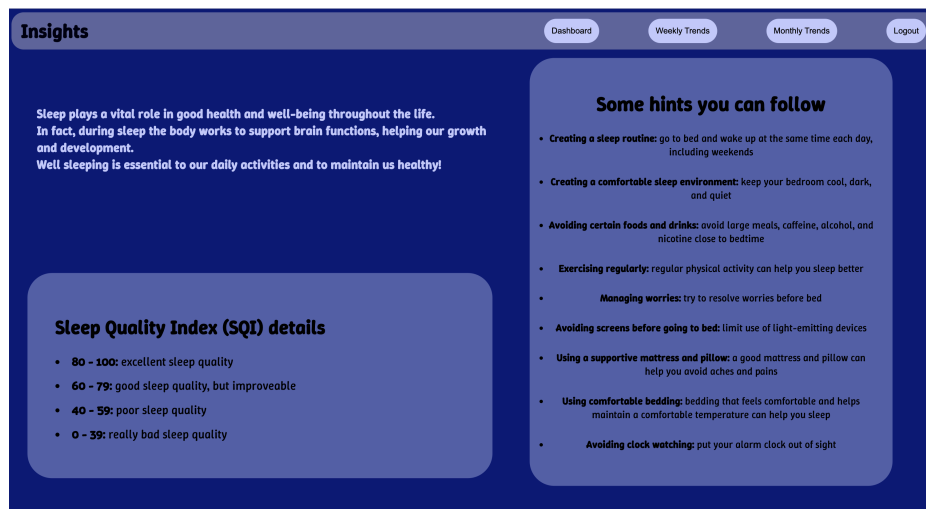
Come accennato, dalla barra di navigazione è possibile muoversi tra le funzioni complementari di Somni.

Si parte dall'analisi della sezione Insight, che svolge una funzione informativa per l'utente, mostrando l'importanza di un buon sonno, illustrando alcuni suggerimenti utili per poter migliorare il proprio riposo e suggerendo un'interpretazione dei diversi livelli dell'indice di qualità del sonno.

Anche in questa sezione, così come in quelle successive, rimane visibile la barra di navigazione, che consente di tornare alla dashboard e muoversi verso le aree non ancora esplorate.

Si tratta di una componente caratteristica dell'intera applicazione e che rimarrà quindi presente anche nelle sezioni successive.

Resta naturalmente sempre presente il pulsante per il logout, in modo tale che sia accessibile da qualsiasi punto dell'app.



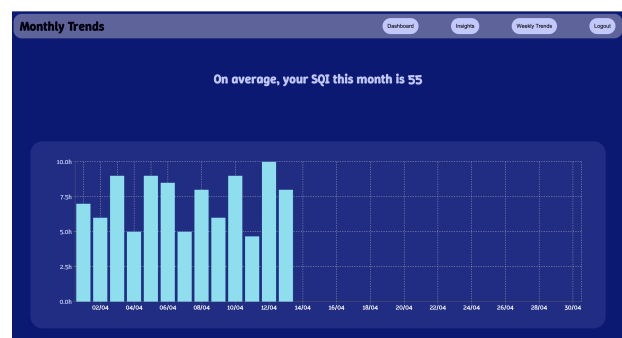
Insights

Le rimanenti sezioni *Weekly Trends* e *Monthly Trends*, invece, accomunabili per estetica e funzionalità, presentano grafici a barre che illustrano gli andamenti del sonno relativi alla settimana e al mese in corso, elaborati sulla base dei dati disponibili.

Oltre alla visualizzazione dei grafici, viene indicato l'indice *SQI* (*Sleep Quality Index*) medio registrato fino a quel momento nel corso della settimana e del mese.



Weekly Trends



Monthly Trends

Si precisa infine che l'applicazione è concepita per dispositivi con una dimensione dello schermo minima corrispondente a 1140 * 960, essendo quindi adatta per tablet (in orientamento orizzontale) e computer.

STRUTTURA BASE - BACKEND:

Concentrandosi invece sulla struttura sottostante, Somni adotta la prassi tipica dello sviluppo in React, puntando quindi su componenti modulari richiamabili in diversi punti dell'applicazione.

In primo luogo, per garantire il corretto funzionamento dell'applicazione, è necessario interfacciarsi con un database su cui poter caricare e recuperare i dati per l'esecuzione delle analisi.

Per questo progetto, ho scelto *Firestore* di Google come supporto per la memorizzazione degli utenti e dei dati ad essi associati, ossia il database contenuto in *Firebase*.

Viene poi utilizzato il modulo *Authentication*, sempre contenuto in *Firebase*, per la gestione degli utenti.

In merito alla gestione dei dati e alla loro organizzazione, si specifica che per gli utenti è stata creata una raccolta denominata *"users"*. Tale raccolta contiene i dati di ciascun utente, archiviati in documenti identificati con l'ID assegnato da Firebase. Ogni utente è caratterizzato da un campo *"email"*, un campo *"nome"*, un campo *"cognome"* e un campo *"data di creazione"*, che consente di tracciare la data di iscrizione dell'utente appunto.

In riferimento ai dati del sonno, questi vengono archiviati in una raccolta denominata *"sleepData"*. Tale raccolta organizza i dati in base all'utente che li inserisce nel sistema, consentendo un filtraggio per utente durante l'utilizzo dell'applicazione. Questo meccanismo impedisce l'accesso ai dati del sonno da parte di utenti non autorizzati, garantendo la tutela della privacy.

Ad ogni utente sono associate delle voci caratterizzate da un timestamp. Ciascuna voce contiene i campi relativi alla data di creazione, alla fase del sonno, al **timestamp** e all'identificativo dell'utente, al fine di assicurare che ogni valore raccolto sia effettivamente associato all'utente connesso.

A livello architetturale, ho definito un file JavaScript denominato *"firebase-config"* per la configurazione del database all'interno di Somni e un file *"get-db-data"* che definisce la funzione *"getSleepData"*. Quest'ultima consente di recuperare i dati nell'applicazione.

Nello specifico, questo componente viene richiamato in ogni pagina principale dell'applicazione tramite l'utilizzo dell'hook *"useFetch()"* per recuperare i dati dal database. I dati vengono successivamente formattati in modo differenziato in base alle modalità di elaborazione e presentazione all'utente.

È fondamentale che l'utente abbia effettuato l'accesso alla piattaforma prima di poter visualizzare qualsiasi dato. A tal fine, è stato definito il componente *"auth-manager"*, contenente le direttive per la gestione di registrazione, login e logout. Le sezioni di Somni sono accessibili solo previa verifica dell'utente connesso. Pertanto, non è possibile accedere ad una pagina senza aver prima effettuato l'accesso dalla schermata di benvenuto.

Come ripetutamente menzionato, Somni è un'applicazione web strutturata in più sezioni, ciascuna corrispondente a una pagina specifica. Di fondamentale importanza è quindi il concetto di navigazione, implementato tramite il noto framework *React Router*. Tale framework, definito nel file *main.jsx*, ovvero il nodo principale dell'applicazione React, specifica tutte le rotte per le diverse pagine, nonché l'entry point; corrispondente alla pagina di benvenuto.

Il funzionamento di Somni si basa sulla presenza di dati coerenti, organizzati secondo le modalità precedentemente definite.

Il componente "*csv-uploader*" gestisce il caricamento dei dati da file CSV.

Si accede quindi al database Firestore e si utilizza la libreria *Papaparse* per il parsing e il caricamento dei dati dai file CSV, previa verifica della loro consistenza, che devono contenere un timestamp e uno *sleepStage* per ogni entry.

I restanti campi, precedentemente illustrati, vengono aggiunti da questo componente per assecondare la logica dell'applicazione.

Viene eseguito un controllo sui dati caricati in modo da evitare duplicati, che verrebbero scartati.

Come indicato, ogni pagina presenta una barra di navigazione che sfrutta le rotte definite in React Router per indirizzare l'utente verso la pagina desiderata.

La pagina più complessa è indubbiamente la Dashboard, la quale si occupa della presentazione di molteplici dati.

Essa si caratterizza per la presenza di diversi componenti aggregati, ciascuno correlato ai componenti specificati nella sezione relativa al frontend.

Al momento dell'avvio della pagina, si esegue il fetching dei dati, che vengono successivamente trasmessi ai moduli responsabili della generazione dei grafici. Ciascun grafico formatta i dati in base alle proprie esigenze, aggiungendo campi come la durata, utile per il calcolo del tempo totale trascorso a letto e nelle diverse fasi del sonno (sfruttando come principio base il fatto che ogni voce nei file CSV rappresenta un minuto della notte analizzata).

Una volta formattati i dati, la libreria scelta per la rappresentazione grafica, ovvero *Recharts*, è in grado di generare il grafico a torta, il grafico dell'ipnogramma e, successivamente, seguendo la medesima logica, i grafici a barre, questi ultimi contenuti nelle pagine dei Trend.

I dati vengono comunque filtrati, prima di essere passati ai grafici, in modo da poter trattare solo quelli relativi al giorno selezionato dall'utente.

Un secondo filtraggio viene eseguito per calcolare la durata di ogni fase del sonno, al fine di generare il valore del SQL (Sleep Quality Index) visualizzato all'utente.

Analogamente, nelle sezioni dedicate ai Trend, viene proposta un'analisi aggregata degli SQL dei giorni registrati, fornendo una media dei valori.

Tutte le pagine sono progettate per comunicare all'utente un messaggio durante il recupero dei dati, in modo da informarlo che l'applicazione sta recuperando ed elaborando le informazioni necessarie.

GENERAZIONE DEI DATI:

Il team di IoT4Care ha fornito un file CSV di esempio per guidare lo sviluppo dell'applicazione e delle sue funzionalità. Tuttavia, il file conteneva dati riferiti a un unico giorno dell'anno, risultando inadeguato per soddisfare le esigenze di Somni, che necessita di elaborare informazioni su larga scala e in tempo reale, relative ai giorni della settimana corrente e al mese in corso.

Per ovviare a questa limitazione, è stata sviluppata un'applicazione dedicata, gestita da riga di comando e denominata *"data-generator"*, scritta in linguaggio C. Tale applicazione interagisce con l'utente, richiedendo gradualmente le informazioni necessarie per la generazione dei dati.

Inizialmente, viene richiesto all'utente di indicare il primo giorno della settimana corrente, facendo riferimento al lunedì come da convenzione europea. Successivamente, si avvia un ciclo *for* organizzato in sette step, uno per ciascun giorno della settimana, che richiede all'utente di specificare le ore trascorse a letto e l'orario in cui si corica.

Il programma accede quindi alla directory *"data"* e inserisce i risultati prodotti in file specifici, denominati secondo il formato *"sleep-data-giorno_della_settimana-numero_del_giorno.csv"*.

I dati prodotti non riflettono l'effettiva distribuzione delle fasi del sonno di una reale sessione di sonno, presentando i momenti delle diverse fasi in modo discontinuo. Ciò è dovuto al fatto che, sebbene siano imposti dei vincoli circa la transizione delle fasi, queste vengono comunque selezionate in modo pseudocasuale. Di conseguenza, si assiste a un ipnogramma piuttosto irregolare, dettaglio tuttavia trascurabile considerando che lo scopo unico di questo programma ausiliario è quello di fornire dati d'esempio per illustrare il corretto funzionamento di Somni.

NOTE AGGIUNTIVE:

Inizialmente, il reload della pagina comportava una latenza significativa prima della visualizzazione dei dati, dovuta alla necessità di React di eseguire nuovamente il recupero dei dati ad ogni caricamento.

Per risolvere il problema, si è fatto ricorso all'utilizzo di *localStorage*: dato che l'unico momento in cui è possibile modificare i dati è prima dell'accesso alla dashboard, quindi nel momento in cui si possono caricare nuovi dati mediante file CSV, è lecito e opportuno salvare tali dati in un archivio locale, al fine di velocizzare eventuali operazioni di aggiornamento da parte dell'utente.

Si registra un'ottimizzazione significativa.

Altra problematica inizialmente affrontata consisteva nella sopravvivenza delle informazioni di login nello spostamento tra le diverse pagine, protette come si ricorda, nonché nell'esecuzione di un refresh della pagina.

Ad ogni caricamento risultava impossibile accedere, in quanto non veniva salvato lo stato dell'utente. È quindi stato introdotto un *contesto*, che ha proprio lo scopo di salvare queste informazioni e condividerle tra i componenti, consentendo una navigazione fluida e senza compromessi tra le diverse pagine.